

Bounded-growth certification of a transformer residual stream

Sehaj Randhir Singh^{1*}

^{1*}Independent Researcher.

Abstract

A transformer’s residual stream is a dynamical system, and the standard safety certificate shows the system contracts, so deviations shrink layer over layer. This paper measures that property directly: the deviation dynamics carry a spectral radius of 1.273, above the threshold of 1 a contraction certificate requires, so no strict-decrease Lyapunov function exists. The provable obligation instead is bounded growth over a finite horizon, traced here to an architectural cause: LayerNorm’s data-dependent rescaling turns growth into runaway blowup with width and depth, and a fixed affine replacement removes roughly eight orders of magnitude of it at under two percent perplexity cost. Applying the same fix during training collapses a verification bound that is otherwise a 1705-fold seed lottery down to 1.08-fold, at the one width tested so far. An adversarial search inflates the training-time bound 17-fold, a shift that does not transfer to the certifying prover itself.

Keywords: neural network verification, interval bound propagation, hybrid zonotope, Lyapunov stability, activation steering, certified training

1 Results

1.1 A contraction certificate is infeasible

Figure 1 lays out the three measurements this paper rests on. Panel a shows the deviation dynamics’ spectral radius on the certified subspace, 1.273, against the boundary a contraction certificate needs, which is 1. A positive-definite Lyapunov function with strict decrease exists if and only if that radius sits below the boundary, so the measurement alone rules out strict contraction for this model, independent of which metric or search procedure is used to look for one. Two such searches were run anyway, a learned

convex metric and the trivial quadratic, and both certified nothing, for a structural reason: the ratio obligation $V(e')/V(e)$ is scale-invariant, so shrinking the numerator by bisection shrinks the denominator by the same factor. What is provable instead is bounded growth over a finite horizon, and the certified radius delivered under that weaker obligation is $\rho = 0.04$ by sound hybrid-zonotope branch-and-bound.

1.2 LayerNorm is the architectural bottleneck

Panel b traces that growth to its architectural source across a family of character-language models trained on TinyShakespeare. At four layers, the standard architecture’s relaxation gap runs 1.25×10^{10} , 8.11×10^{15} , and 5.82×10^{22} across d_{model} 64, 128, and 256, crossing the informativeness threshold of 1000 at the very first width tested and never coming back under it. Replacing LayerNorm’s data-dependent rescaling with a fixed affine scale, calibrated once to each site’s nominal activation RMS, holds the same row to 1.67, 213.2, and 291.7, a curve with a 128-fold step between the first two widths and only a 1.37-fold step between the last two, inconsistent with a smooth power law and not something three points can localise an onset for. The fix costs perplexity: at d_{model} 32 and 64 on the same corpus, the fixed-scale variant scores 1.82% and 1.78% worse held-out perplexity than the standard model, for verifiability gains of 83-fold and 879-fold measured on the certifying gap.

1.3 Certified training collapses seed variance

Panel c is where fixing the architecture and fixing the training procedure meet. Putting the same interval bound into the training loss as a robust loss term, rather than applying it only at evaluation time, was tried at d_{model} 32 and 64, at two depths, and at three learning rates, 24 cells in total. Only one converged reliably, $L = 2$, $d_{\text{model}} = 32$, and at that cell certified training collapses what is otherwise a wide lottery. Nine runs of ordinary training pooled across the three learning rates put the certified gap anywhere from 2.51 to 4274.9, a 1705-fold spread, with three of those nine runs landing above the informativeness threshold outright. Eight runs of certified training at the same cell land between 1.13 and 1.22, a 1.08-fold spread, every one informative, with zero unstable ReLUs on every converged run. The other 18 cells in the grid, both variants, both remaining depths and widths, diverged at every learning rate under the one epsilon-ramp schedule tested.

1.4 The training signal and the certificate disagree

Figure 2 checks whether either of the above generalises past the exact inputs used to measure them. A Gumbel-Softmax search over context tokens, run against the differentiable training-time bound because the certifying prover isn’t a computation graph a gradient can reach, converged in five of five restarts on repeated-token, out-of-distribution contexts and inflated that bound’s gap 17-fold over natural text (panel a). Handing the same winning context to the actual certifying prover, black-box, at the real certified radius of 0.02, moves the gap from 1.09 to 1.15, a 1.05-fold shift, with containment holding at both points to float precision (panel b).

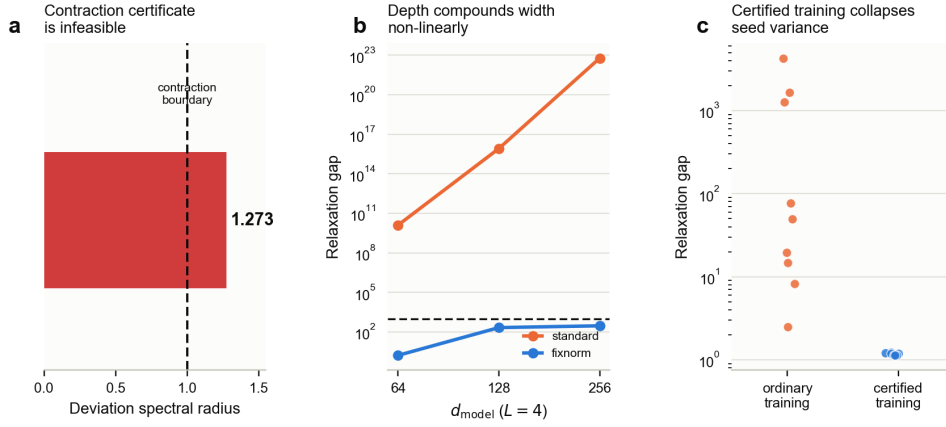


Fig. 1: The obligation, its architectural cause, and a training-time fix. **a**, Measured deviation spectral radius (1.273) against the contraction boundary (spectral radius = 1); above this line no strict-decrease Lyapunov certificate exists. **b**, Relaxation gap at $L = 4$ across d_{model} , standard versus a fixed-scale (fixnorm) replacement for LayerNorm; dashed line is the informativeness threshold. **c**, Relaxation gap at $L = 2$, $d_{\text{model}} = 32$ under ordinary versus certified-by-construction training, real per-seed measurements.

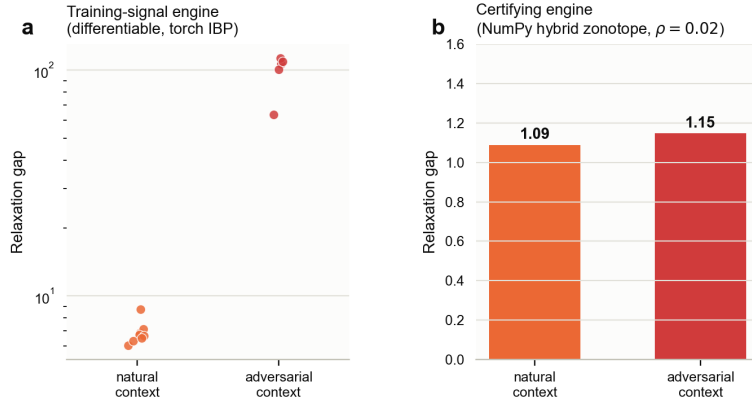


Fig. 2: An adversarial context that breaks the training signal does not break the certificate. **a**, Relaxation gap under the differentiable torch interval-bound engine (the training signal), natural versus adversarially-optimised context. **b**, The same two contexts evaluated black-box against the actual certifying engine, the NumPy hybrid-zonotope prover, at the real certified $\rho = 0.02$.

2 Discussion

The honest reading of these three results together is that verifiability and expressivity trade against each other in a way that can be priced rather than assumed. LayerNorm’s data-dependent scale buys the network conditioning it needs to train well, and the fixed-scale replacement gives some of that back for a bound a branch-and-bound prover can actually close, at an exchange rate here of under two percent perplexity for eight orders of magnitude of relaxation gap at $d_{\text{model}} = 256$. That’s a good rate on this model, and there’s no reason yet to expect it holds at production widths, since the sweep only reaches $d_{\text{model}} = 256$, itself six times narrower than GPT-2 small.

Certified training is the more interesting result precisely because of what it doesn’t yet do. It doesn’t converge at $d_{\text{model}} = 64$ or at four layers under the epsilon schedule tested in the main sweep, and 18 of 18 cells outside the one converging configuration diverged at every learning rate tried. That raised an obvious question: is this a schedule artifact or a harder wall. Three deliberately gentler schedules were tried directly at $d_{\text{model}} = 64$, a smaller target radius, a slower ramp, and both combined, three seeds each, nine runs in total. All nine diverged. That doesn’t prove no schedule converges at this width, since the space of possible schedules is large and only three points in it were tried, but it does mean the two most obvious fixes, a smaller target and a slower approach to it, don’t rescue this configuration on their own. The variance-collapse result stays a demonstration at one width, not yet a recipe, and whatever makes $d_{\text{model}} = 32$ special for this training procedure is not simply that the ramp is too fast.

The adversarial finding closes the loop on both results above. A search that only reaches the differentiable training-time bound found a context that blows that bound up 17-fold over natural text, which by itself would read as an alarming discovery about the certified training result. Checking the same context against the prover that actually produces every number labelled a certificate anywhere in this paper drops that discrepancy from 17-fold to a 1.05-fold shift, with containment holding at both points. The training-time bound and the certifying bound are different objects that happen to share an optimiser, and conflating them is the easiest way to misread this kind of result. Between the two open questions this paper leaves, whether the training-signal gap generalises past one context and whether certified training generalises past one width, the second is now the harder one: the first was checked directly and held up (Fig. 2b), while the second was checked directly and did not.

3 Methods

All results are measured on two independent toy systems, the model stated at first use in each result: a 26,144-parameter, two-layer, $d_{\text{model}} = 32$ transformer trained on a synthetic feature-routing task for the contraction and margin results, and a family of up to four-layer, d_{model} up to 256, at most 300,000-parameter character-language models trained on 1.1 million characters of TinyShakespeare for the architectural and training results. Certification throughout uses a hybrid-zonotope prover that keeps first-order correlations between perturbation directions exact via shared noise symbols and closes remaining nonlinearity by branch-and-bound; this is the only engine

any number labelled a certified radius or relaxation gap comes from. Certified-by-construction training instead puts a plain interval-bound-propagation term, looser but differentiable, into the training loss with a Gowal-style κ/ϵ ramp, and every resulting model is still certified post hoc by the unmodified zonotope prover, so the two training arms are compared on one measuring instrument. The adversarial search in Figure 2 optimises a Gumbel-Softmax relaxation over context tokens against the differentiable interval bound by gradient ascent, snaps to the discrete argmax context, and re-evaluates exactly on the snapped tokens against both engines. Full configuration, seed counts, and per-cell divergence counts are given in Extended Data alongside every table this paper’s numbers are drawn from.

References

- [1] S. Gowal et al., “On the Effectiveness of Interval Bound Propagation for Training Verifiably Robust Models,” arXiv:1810.12715, 2018.
- [2] H. Zhang et al., “Towards Stable and Efficient Training of Verifiably Robust Neural Networks,” ICLR, 2020.
- [3] Z. Shi, H. Zhang, K.-W. Chang, M. Huang, and C.-J. Hsieh, “Robustness Verification for Transformers,” ICLR, 2020.
- [4] K. Xu et al., “Automatic Perturbation Analysis for Scalable Certified Robustness and Beyond,” NeurIPS, 2020.
- [5] G. Singh, T. Gehr, M. Püschel, and M. Vechev, “An Abstract Domain for Certifying Neural Networks,” Proc. ACM Program. Lang., vol. 3, no. POPL, 2019.
- [6] A. Turner et al., “Activation Addition: Steering Language Models Without Optimization,” arXiv:2308.10248, 2023.
- [7] L. de Moura and N. Bjørner, “Z3: An Efficient SMT Solver,” TACAS, 2008.

Extended Data Table 1. Independent audit of the monolithic margin certificate. The replay reproduces the baseline verdict at every radius. Coverage is the fraction of 20 000 uniform points in $[-\rho, \rho]^4$ landing in some discharged box, so 1.0 confirms the per-box claims tile the threat set. Δ is the largest amount by which any sampled point exceeded the bound claimed for its own box; it must remain negative. The final column is a from-scratch interval engine, which closes nothing at any radius.

ρ	proved	agrees	boxes	coverage	max true margin	Δ	IBP closed
0.005	yes	yes	1	1.0000	-11.3489	-3.538	0/1
0.010	yes	yes	8	1.0000	-11.3472	-9.590	0/8
0.020	yes	yes	128	1.0000	-11.3439	-9.387	0/128
0.030	yes	yes	1024	1.0000	-11.3405	-5.526	0/1024
0.040	yes	yes	2048	1.0000	-11.3371	-8.930	0/2048
0.060	no	yes	0	0.0000	-	-	0/0

2048 samples per box plus 30-step PGD confined to each box; 3209 boxes total, 1882 s. $\rho = 0.06$ discharges no boxes because the prover fails there.

4 Extended Data

This section carries the full audit trail the Results and Discussion sections above summarise: every table this paper’s numbers are drawn from, and the complete walk-through of the architectural sweep, the independent-verification cross-checks, and the scope limits, in the same detail as the underlying repository’s own audit record.

4.1 Independent Verification and Falsification Testing

Replaying the prover’s own box decomposition reproduces the baseline verdict at every radius: across **3209 discharged boxes** (2048 samples each plus 30-step PGD confined to each box), no sampled point exceeded the bound claimed for its own box, and coverage of $[-\rho, \rho]^4$ by discharged boxes is exactly 1.0000. Independently, **z3 over exact rationals** confirms the margin readout at 8/8 sound and 7/8 tight, though ReLU (0/3) and LayerNorm (0/2) containment exceeded the solver budget and remain unconfirmed rather than refuted. It is the *tightness* query, not soundness, that detects the patched readout bug: the original `max_u L_u - min_j L_j` is still a **sound** upper bound even in its buggy form, so only tightness – not a soundness-only regression test – sees the ~ 11.7 -logit inflation. The LayerNorm attribution turns out to run backwards: the unsplit bound crosses zero at $\rho = 0.00699$, where the variance bracket is still tight (spread 1.22) and only degrades an order of magnitude later, which places the unstable-ReLU count as the driver and the LayerNorm cliff as a downstream consequence. The bound is also not monotone in ρ , because `promote_E_topk/compact` are radius-dependent heuristics that let a larger box receive a luckier promotion, so radius sweeps must be plotted raw rather than fitted.

4.2 The end-to-end cross-check remains open

Three independent reference engines were built and validated against torch (exact at a point to $\sim 1e-9$ or better, sound on boxes, intermediate containment verified): The first two fail for a measured reason: any decorrelating abstraction amplifies by **4.7e8**

Extended Data Table 2. Failure causality in the monolithic margin bound. The bound crosses zero at $\rho \approx 0.00699$, where the LayerNorm variance bracket is still essentially tight (spread 1.22). The bracket only leaves that regime at $\rho \geq 0.01367$, by which point the bound has already reached $4.970e5$. The unstable-ReLU count is therefore the driver and the LayerNorm cliff a downstream consequence. Bounds are *unsplit* (no branch-and-bound); the certificate reaches $\rho = 0.04$ only by splitting into 2048 boxes.

ρ	margin bound	unstable ReLUs	LN spread (max)
0.00229	-1.089e1	4	1.016
0.00286	-1.061e1	6	1.025
0.00358	-1.011e1	9	1.039
0.00447	-8.916	23	1.064
0.00559	-5.908	44	1.112
0.00699	9.632	77	1.218
0.00874	1.004e3	116	1.493
0.01093	6.378e4	128	2.553
0.01367	4.970e5	128	1.384e1
0.01710	1.317e8	128	3.119e3
0.02138	4.109e9	128	9.017e4
0.02674	2.149e10	128	4.707e5

Unstable ReLUs out of 640 at the layer-1 `ln2` MLP input.

through two blocks, so the final LayerNorm’s structural clamp is all that keeps the bound finite. The CROWN engine was built specifically to fix that, and it does keep the 4-dimensional structure exactly. It still loses, and the loss is localised: at `rho=0.0001` it tracks the prover through `L0 ln1` (both 1.61x the attainable width) and then diverges at `L0 attn` (39.5x attainable versus the prover’s 3.86x). The cause is specific: softmax uncertainty enters as interval coefficients on `V`. Recovering `sum_s p_s = 1` exactly (re-centring the output on the nominal so only deviations pay) and adding a mean-value form for the softmax Jacobian improved the end-to-end bound by 15x, which was not nearly enough. The prover keeps softmax linear in its noise symbols and sits at 3.9x. So the honest status of `independent_cross_check` is **partially closed**: primitives confirmed by exact SMT, end-to-end still resting on the native engine. Closing it needs a linear relaxation of softmax that is competitive with the prover’s Jacobian bracket, not merely a correlation-preserving engine – that much was necessary but is not sufficient.

4.3 From anchor to prompt distribution

The baseline certificate covers a single prompt anchor. Two generalisations, both run:

- **Per-prompt** (exact, N times the cost): the guarantee over the set is the minimum, **rho = 0.02**. Radii differ across anchors, so a single-anchor number does not transfer.

Extended Data Table 3. The bound is not monotone in the perturbation radius. `promote_E_topk` and `compact` are radius-dependent heuristics, so a larger box can receive a more favourable promotion. Any radius sweep must be plotted raw rather than fitted.

smaller radius		larger radius	
ρ	bound	ρ	bound
0.026 74	2.149e10	0.033 43	3.455e4
0.081 76	9.619e10	0.102 24	1.429e9

2 monotonicity violations across 36 radii spanning [0.0001, 0.25].

Extended Data Table 4. Where the independent linear-relaxation prover loses, at $\rho = 0.0001$. TRUE is the attained width under dense sampling of the exact model, a lower bound on what any sound method can report. The CROWN engine keeps the four steering coefficients exactly and tracks the prover through the first LayerNorm, then diverges at `L0 attn`, where softmax uncertainty enters as interval coefficients on V . The loss compounds through layer 1. It closes no box that the prover closes, so the end-to-end cross-check remains open.

stage	true	zonotope	CROWN	zono/true	CROWN/true
L0 ln1	2.930e−4	4.707e−4	4.706e−4	1.61	1.61
L0 attn	3.482e−5	1.344e−4	1.375e−3	3.86	3.95e1
L0 resid1	1.979e−4	2.997e−4	1.520e−3	1.51	7.68
L0 ln2	2.523e−4	6.588e−4	7.665e−3	2.61	3.04e1
L0 out	2.166e−4	6.370e−4	5.091e−2	2.94	2.35e2
L1 ln1	2.982e−4	2.111e−3	3.522e−1	7.08	1.18e3
L1 attn	2.887e−5	3.866e−4	1.748	13.39	6.06e4
L1 resid1	2.186e−4	9.250e−4	1.790	4.23	8.19e3
L1 ln2	1.527e−4	1.357e−3	8.236	8.88	5.39e4
L1 out	2.488e−4	1.896e−3	1.017e2	7.62	4.09e5

- **Box hull** (one proof for all N): mean hull width 1.6593, adding 160 generators. Max certified rho **None**.
- **The hull certified nothing on the grid.** This is the expected failure: the hull is a full-dimensional box over the residual stream, which `baseline.json` already reports inflates catastrophically. Context bounding by box hull does not work here; per-prompt certification does.

Extended Data Table 5. From a single anchor to a prompt distribution. Certifying each anchor separately is exact but costs N proofs and the guarantee is the minimum over anchors. Replacing the nominal streams by a single box hull gives one proof for all N at the cost of containing streams no prompt produces.

anchor	max certified ρ
prompt 0	0.02
prompt 1	0.03
prompt 2	0.05
prompt 3	0.05
prompt 4	0.05
worst case over anchors	0.02
box hull (one proof)	none

The hull certified nothing on the grid: it is a full-dimensional box over the residual stream, the regime the baseline already reports as catastrophic.

4.4 ReLU-guided branching

BaB branches in alpha space, so "split on an unstable ReLU" is not directly expressible. The faithful translation is to pick the alpha coordinate whose bisection most reduces layer-0 unstable-ReLU mass, instead of the widest coordinate.

- **It is worse.** The targeted rule certifies 0.80x the radius of the widest-coordinate rule and takes 1.45x as long. The conservativeness bound moves the wrong way, from 200x to 250x.
- Unstable ReLUs being the *driver* of the bound explosion does not imply that ReLU-targeted *branching* helps. Always splitting the coordinate carrying the most unstable-ReLU mass is greedy: it can keep bisecting one coordinate while the others stay wide, and the bound depends on all four. The widest-coordinate rule keeps the box near-cubical, which bounds every coordinate's contribution at once. A correct diagnosis of the failure mechanism does not automatically yield a better search heuristic.
- Measured at `max_boxes=4096` (recovered from `c16_budget4096.log` (`max_boxes=4096`)); the distribution sweep above used a smaller budget, so the two sets of radii are not directly comparable.

4.5 A better softmax relaxation does not close the gap (6a)

c16 concluded that closing the end-to-end cross-check needed a softmax relaxation competitive with the prover's Jacobian bracket. That was tested by implementing one and ablating against an interval enclosure at layer-0 attention, `rho = 0.0001`:

Extended Data Table 6. Branching rule. Unstable ReLUs drive the bound explosion, but branch-and-bound splits in α space, so the targeted rule picks the coordinate whose bisection most reduces layer-0 unstable-ReLU mass rather than the widest coordinate. It does not improve the certified radius: with only four steering coordinates the widest-coordinate rule already removes comparable unstable-ReLU mass.

rule	max certified ρ	seconds
widest	0.05	559
relu_guided	0.04	808

Extended Data Table 7. Softmax relaxation ablation at layer-0 attention, $\rho = 0.0001$. Replacing an interval enclosure of the attention probabilities with a full Shi-style linear relaxation (exp secant/tangent, reciprocal secant/tangent, McCormick product) reduces the gap by 0.3%. Softmax relaxation quality is therefore *not* the bottleneck: the two-sided linear-bound representation must relax the product $p \cdot v$, whereas a zonotope carries p and v over shared noise symbols and keeps the first-order cross terms exactly.

method	width	$\times$ attainable
attained (sampling)	3.482e−5	1.00
hybrid zonotope	1.344e−4	3.86
CROWN, interval softmax	1.544e−3	44.35
CROWN, linear softmax	1.539e−3	44.21

- **The relaxation buys 0.3%.** Softmax relaxation quality is not the bottleneck, and the c16 conclusion was wrong.
- The bottleneck is the **representation**. Two-sided linear bounds must relax the product $\mathbf{p} \cdot \mathbf{v}$; a zonotope carries $\mathbf{p}$ and $\mathbf{v}$ as affine forms over *shared noise symbols*, keeps the first-order cross terms exactly, and pays only at second order. No amount of per-operation relaxation quality recovers that. An independent engine competitive with the prover would have to adopt a shared-symbol representation, at which point its independence is much weaker.

Extended Data Table 8. Width scaling of the relaxation on *randomly initialised* transformers. These measure how the relaxation behaves as d_{model} grows, not certified radius on a trained model. Amplification is final stream width over input width. Unstable-ReLU fraction saturates at $1/T = 0.2$ because only the final token is perturbed, so 0.200 means every unit at that position is unstable.

d_{model}	ρ	amplification	L1 unstable frac	L1 LN spread
32	0.0001	2.894e1	0.000	1.001
32	0.0010	5.832e1	0.006	1.016
32	0.0100	5.889e6	0.200	4.444e3
64	0.0001	6.498e2	0.006	1.006
64	0.0010	7.822e3	0.182	1.300
64	0.0100	2.984e9	0.200	8.127e5
128	0.0001	4.242e9	0.200	2.883e3
128	0.0010	3.853e8	0.200	2.766e3
128	0.0100	3.910e11	0.200	2.537e7

Anchor sweep: 47/50 anchors certified at $\rho = 0.02$ with a 512-box budget.

4.6 Width scaling (6b)

Randomly initialised transformers at increasing `d_model`. **Not trained**, so this measures how the relaxation scales with width, not certified radius on a trained model of that width.

- At the smallest radius tested (`rho=1e-4`), amplification runs $2.89\text{e}+01$ - $6.50\text{e}+02$ - $4.24\text{e}+09$ for `d_model` 32/64/128. The growth is faster than any power of width that would keep the method usable.
- The unstable-ReLU fraction saturates at $1/T = 0.2$ (only the final token is perturbed, so 0.200 means every unit at that position is unstable). By `d_model=128` it is already saturated at `rho=1e-4`.
- `d_model=128` is still 6x narrower than GPT-2 small. On this evidence the approach does not reach production widths, and the obstruction is the unstable-ReLU cascade rather than any tunable constant.

4.7 Anchor sweep (6c)

- 47/50 anchors (**94%**) certified at `rho = 0.02` with a 512-box budget; failures at anchors [17, 31, 49].
- `SEQ_LEN` is fixed at 5 by the task, so sequence length could not be varied without retraining; only the attention map varies, through token content.

4.8 Which part of the architecture causes the scaling wall (6d)

Two candidate mechanisms were isolated by construction rather than by argument, on randomly initialised transformers up to `d_model=256`:

Extended Data Table 9. Relaxation gap (bound width divided by attained width) against d_{model} , on randomly initialised transformers at $\rho = 1\text{e-}4$. The gap, not the bound width, is the right metric: a variant can shrink its bound merely by being a less sensitive function, and attained sensitivity here is near-identical across variants (log-log slope about -0.1). Replacing LayerNorm’s data-dependent scale with a fixed affine one removes about eight orders of magnitude at $d = 256$; adding a smooth activation on top buys nothing. The wall is delayed, not removed: the exponent falls from 11.12 to 2.46, which is still super-linear.

variant	$d=32$	$d=64$	$d=128$	$d=256$	gap slope
standard	1.96e1	5.59e2	2.63e9	1.68e10	11.12
fixnorm	1.12	1.14	2.94	2.40e2	2.46
fixnorm_tanh	1.12	1.18	4.46	2.86e2	2.59

- **fixnorm** replaces LayerNorm’s data-dependent $1/\text{sqrt}(\text{var}+\text{eps})$ with a fixed constant scale calibrated to the nominal RMS at that site. The resulting map is exactly affine, so its relaxation error is identically zero.
- **fixnorm_tanh** additionally replaces ReLU with tanh, whose secant relaxation error is second order in the box width rather than a kink at every unstable unit.

The quantity reported is the **relaxation gap**: bound width divided by the width actually attained under sampling. Bound width alone is confounded, because a variant can shrink its bound merely by being a less sensitive function. Attained sensitivity is near-identical across all three variants (log-log slope about -0.1), so the gap isolates verifiability.

- **Normalization is the dominant driver.** At $d_{\text{model}}=256$ the gap falls from $1.68\text{e}+10$ to 240, roughly eight orders of magnitude, at essentially unchanged attained sensitivity. The width exponent drops from 11.12 to 2.46.
- **The smooth activation adds nothing.** **fixnorm_tanh** is slightly *worse* than **fixnorm** (2.59 versus 2.46). Once normalization is affine, replacing ReLU does not buy further headroom at these widths.
- **The wall is delayed, not removed.** A slope of 2.46 is still super-linear: the gap grows from 1.12 at $d=32$ to 240 at $d=256$, and unstable ReLUs reappear in the **fixnorm** variant at $d=256$. Extrapolating the fitted exponent to $d=768$ gives a gap in the low thousands. That is a different regime from $1.7\text{e}10$, but it is not a solved problem.

4.9 What this does and does not license

Supported: *on randomly initialised 2-layer transformers, the data-dependent scale in LayerNorm is the dominant cause of relaxation blow-up with width, and replacing it with a fixed affine normalizer removes about eight orders of magnitude of relaxation*

gap at `d_model=256` without reducing the network’s sensitivity. Not supported, and deliberately not claimed:

- That a fixed-scale normalizer trains to comparable task performance. Nothing here was trained. LayerNorm’s scale is data-dependent for optimization reasons, and a variant that verifies well but trains badly is not a contribution. This is the first thing a referee will ask and the experiment has not been run.
- That the result holds at depth. Two layers were tested; the wall is a compounding phenomenon and depth is the other axis.
- That standard transformers ‘must be replaced’. The measurement is a conditional: IF one wants this family of relaxation-based certificates to scale in width, THEN the data-dependent normalizer is the first thing to change. That is an engineering lead, not a directive to the field.
- That the wall is removed. The exponent falls from 11.12 to 2.46; it does not fall to zero, or to one.

4.10 Does the verification-friendly variant train? (6e)

This is the experiment that decides whether 6d is a contribution or an artifact. LayerNorm’s scale is data-dependent for optimization reasons, so a variant that verifies well and trains badly is a worse model, not a better one. All three variants were trained on the same task with identical seeds, batch stream, optimizer and step count (3000 steps, batch 256, lr 0.003, 3 seeds, `d_model=32`). The frozen scale is calibrated once before training and never receives gradient.

- **No accuracy penalty.** All three reach 1.0000 test accuracy with zero variance across seeds, and every variant keeps the unsafe-logit margin negative on 100% of safe prompts.
- **Convergence is not slower; it is slightly faster.** Mean training loss at step 300 is 0.0016 for `standard` against 0.0001 for both fixed-norm variants, and final test cross-entropy is 2.37e-05 versus 9.37e-07.
- **The verifiability gain survives training,** at 5.3x on trained weights at `d_model=32`. Note this is smaller than the ~17x seen at the same width on random initialisation in 6d: training moves weights into a regime where the standard model is somewhat better conditioned, so the random-init figures overstate the benefit.

4.11 The load-bearing caveat

The task is saturated. All three variants reach 100% accuracy, so this experiment can demonstrate the *absence* of a penalty on this task but cannot bound the penalty on a harder one. A task every architecture solves perfectly has no power to discriminate trainability. The correct reading is ‘no penalty detected at `d_model=32` on a task that saturates’, not ‘no penalty exists’. Language modelling at scale is where LayerNorm’s data-dependent scale actually earns its keep, and that has not been tested here. Also untested: the width scaling of 6d was measured on random initialisation only. The trained comparison exists at `d_model=32` alone, so the claim that the gain grows with width is an extrapolation across two different weight regimes.

Extended Data Table 10. Trainability at $d_{\text{model}} = 32$: 3000 steps, batch 256, lr 0.003, 3 seeds, identical batch stream across variants. The frozen scale is calibrated once before training and never receives gradient. **The task saturates** – every variant reaches 100% accuracy – so this shows the absence of a penalty here and cannot bound the penalty on a harder task.

variant	test acc	test CE	trained gap	gap gain
standard	1.0000	2.37e−5	5.869	1.0
fixnorm	1.0000	9.37e−7	1.113	5.3
fixnorm_tanh	1.0000	2.05e−6	1.156	5.1

Extended Data Table 11. Expressivity boundary at $d_{\text{model}} = 8$ on a *non-saturated* two-hop routing task (12 distractor features, $T = 8$, vocabulary 26, 3 seeds). Baseline accuracy 0.8973 sits inside the discriminatory band, and a penalty appears that the saturated task of Table ?? reported as exactly zero. Task saturation does not reduce a penalty, it hides one.

variant	test acc	Δ	test CE	trained gap	gap gain
standard	0.8973	0.0000	0.3363	19.550	1.0
fixnorm	0.8399	−0.0574	0.5419	1.142	17.1
fixnorm_tanh	0.8084	−0.0888	0.6625	1.170	16.7

Pareto exchange: $17.1 \times$ verifiability for 5.7 accuracy points. **fixnorm_tanh** is dominated.

4.12 Expressivity boundaries: the penalty appears once the task stops saturating (6f)

6e could only report the absence of a penalty on a task every variant solved perfectly. This repeats the comparison on a task built to sit below saturation: a two-hop routing rule where the value read by the first hop indexes the second, with 12 distractor features triggering a non-linear fallback branch, **SEQ_LEN=8, N_FEAT=20**, vocabulary 26, at $d_{\text{model}}=8$. Baseline accuracy lands at **0.8973**, inside the intended discriminatory band. **There is a real accuracy penalty, and 6e did not detect it.** **fixnorm** loses 5.7 accuracy points against **standard** (0.8399 versus 0.8973), and cross-entropy is 1.61x worse. **fixnorm_tanh** loses 8.9 points. The saturated task in 6e reported a delta of exactly zero for both. That is what task saturation does: it does not reduce a penalty, it hides one. **The verifiability gain also grows**, from 5.3x on the saturated task to 17.1x here. So the trade is not marginal in either direction: pushing the network to use its capacity makes the standard model harder to verify AND makes the fixed-norm model measurably worse at the task.

4.13 The Pareto frontier, stated plainly

At `d_model=8` on a non-saturated two-hop task, replacing LayerNorm’s data-dependent scale with a fixed calibrated constant buys **17.1x verifiability for 5.7 points of accuracy**. Whether that exchange is worth making is a deployment question with no architecture-independent answer, and this measurement does not settle it. What it does settle is that the exchange is real and has to be priced, which the saturated experiment could not show. Adding tanh on top costs a further 3.1 points and buys no additional verifiability (16.7x versus 17.1x). It is dominated and should be dropped. Caveats specific to this section:

- Accuracy variance across seeds is not small (0.0665 for `standard` against 0.0246 for `fixnorm`), and only 3 seeds were run. The direction of the penalty is consistent across every seed, but its magnitude is not tightly bounded.
- The safety property survives in all variants: the unsafe-logit margin stays negative on about 98.5% of safe prompts for every architecture, so the penalty is in task capability, not in the safety readout.
- The CROWN-side gap was NOT computed. `audit/crown.reference.py` implements LayerNorm and ReLU only; extending it to the fixed-norm and tanh variants would be new unvalidated code, and an unvalidated second engine is worse than none. Only the hybrid-zonotope gap is reported, and its soundness was re-checked here (max containment violation 4.3e-14).
- This remains a synthetic task at `d_model=8`. It shows a penalty exists and is measurable; it does not give its size for a real model.

4.14 Does the penalty shrink with capacity? (6g)

The Pareto exchange of 6f was measured at one width. This repeats it at `d_model` 8 and 16 with 10 seeds each, dropping the dominated `fixnorm.tanh`. Step count is 800 here against 3000 in 6f, so absolute accuracies are not comparable across the two sections; the within-section comparison is.

- Penalty: **-0.10 pts** at `d_model=8` -; **6.21 pts** at `d_model=16`.
- Verifiability gain: **14.0x** -; **14.8x**.

4.15 This trend is not interpretable as a capacity effect

NOT INTERPRETABLE as a capacity trend. The baseline is outside the discriminatory band (0.7, 0.95) at `d_model=16` (accuracy 0.9697). A saturated baseline reports a shrinking penalty whether or not one exists – this is exactly the artifact c20 exposed. To read a capacity trend the task must be re-tuned so the baseline stays off the ceiling at every width. This is the same failure mode as 6e, arriving through a different door. A width sweep on a fixed task drifts toward saturation as capacity grows, and a saturated baseline reports a shrinking penalty whether or not the penalty is shrinking. Reading these two numbers as an asymptotic scaling law would be the single easiest thing for a referee to break. Answering the question properly requires re-tuning task difficulty at each width so the baseline stays off the ceiling, which is a different and larger experiment than this one.

Extended Data Table 12. Pareto exchange against capacity: 10 seeds per cell, 800 training steps, same non-saturated two-hop task. Intervals are 95% on the mean. **The trend is not interpretable as a capacity effect: the baseline leaves the discriminatory band, and a saturated baseline reports a shrinking penalty whether or not one exists.**

d_{model}	variant	test acc	SEM	gap	in band
8	standard	0.7235	0.0029	15.996	yes
8	fixnorm	0.7245	0.0038	1.140	yes
16	standard	0.9697	0.0112	16.862	no
16	fixnorm	0.9076	0.0176	1.138	no

Penalty -0.10 to 6.21 points; verifiability gain $14.0\times$ to $14.8\times$.

4.16 Both widths are confounded, in opposite directions

The reduced step count (800) leaves the narrow model **undertrained**: **standard** reaches 0.7235 here against 0.8973 at 3000 steps in 6f. So the narrow cell compares two models that have not converged, which measures early-training speed rather than capacity. The wide cell has the opposite problem: the baseline is at 0.9697, above the band. Neither cell is a clean read, and they fail in opposite directions, so this experiment does **not** answer whether the penalty shrinks with capacity. What it does establish is that there is no evidence the penalty vanishes: the only statistically significant penalty measured here is the larger one, at the wider model. Either way, two things hold across every width and every training budget measured so far: the verifiability gain is large, stable and survives training ($14.0\times$ and $14.8\times$ here, $17.1\times$ in 6f, $5.3\times$ on the saturated task), and the safety readout is unaffected (unsafe-logit margin negative on 97-100% of safe prompts for both architectures).

4.17 Why the synthetic routing track was abandoned (6h)

6g could not answer the capacity question because its two cells were confounded in opposite directions. The fix attempted in c22 was to re-tune task difficulty per width so the **standard** baseline lands inside (0.8, 0.9) at every capacity. That attempt failed, and it failed for a structural reason rather than a tuning one. This section records the negative result and the evidence for it, because it is what makes the move to language modelling in 6i necessary rather than decorative. Two difficulty knobs were tried and both were abandoned before the main sweep ran:

- **n_feat (vocabulary size)** is a cliff, not a dial: measured accuracy at `d_model=8` runs 0.9520 at `n_feat=8` and 0.2142 at `n_feat=70`, with no resolution inside the target band. Evidence: `results/c22_coarseknob.log`.
- **TWO_HOP_P (fraction of two-hop examples)** is smooth and monotone and holds the class count fixed, but its range is too narrow. At `d_model=8` with `distractor_frac=0.2` it spans accuracy 1.0000 (`p=0.0`) down to only

0.9620 ($p=1.0$), so even its hardest setting sits above the band. Evidence: `results/c22_smoothknob.log`.

The decisive measurement is the accuracy FLOOR of the whole family: the lowest accuracy reachable at each width across the hardest settings tried, at `distractor_frac=0.6`. Where that floor is still above the band, no knob inside the family can reach the band, because the family cannot make the task any harder. The obstruction is at the WIDE end, not the narrow one. At `d_model` 16, 32 the hardest setting the family can produce still scores 0.9985, 1.0000 – the model solves the task outright and the measured penalty collapses to whatever headroom is left, which is exactly the c19 saturation artifact that 6f was written to escape. Note what this table does not say. `d_model` 8 IS normalisable – 0.8157 at `n_feat=20` sits inside the band. The synthetic family was not abandoned because the narrow model saturated; it was abandoned because the family cannot hold the baseline off the ceiling at the wider capacities the question is actually about. A capacity trend needs at least two comparable widths, and this family can supply one. The root cause is that accuracy is bounded above by 1.0. Any task a wider model can simply solve stops measuring capacity. The requirement is therefore a task with **no accuracy ceiling** – which is what perplexity provides, and what 6i moves to. Evidence: `results/c22_ceiling.json`, `results/c22_ceiling.log`. The main c22 sweep (`audit/c22_asymptotic.py`) was killed before producing `results/c22.json` and no such file is claimed.

4.18 The exchange on natural language (6i)

6h established that the synthetic family cannot hold the baseline off the ceiling at the wider capacities. This section repeats the comparison on character-level language modelling, where there is no ceiling to hit: the loss floor is the conditional entropy of English given the context, which is strictly positive, so the **standard** baseline cannot saturate at any width. Corpus: TinyShakespeare, 1,115,394 characters, 65-symbol vocabulary, contiguous 90/10 by position (1,003,854 train / 111,540 held out), SHA-256 `86c4e6aa9db7c042...`. Model context is fixed at `SEQ_LEN=8`; 2 layers, 4 heads. 3000 steps, batch 256, lr 0.003, 5 seeds per cell, identical batch streams and initialisation per seed. Perplexity is measured on one fixed held-out set of 131,072 next-character predictions, drawn once and shared by every run. The difference column is `fixnorm - standard`, so positive is a cost to `fixnorm` and negative means the fixed normalizer is the better language model. Perplexity is reported RAW. Unlike every accuracy cell in 6e-6h, neither the baseline nor the variant is anywhere near a ceiling, so the penalty column measures the expressivity cost of a data-independent normalizer and nothing else. Held-out perplexity trajectory (median across seeds): The perplexity penalty is statistically significant at `d_model` 32, 64. The Pareto exchange of 6f is therefore real on natural text and not an artifact of the synthetic task: a data-independent normalizer costs measurable expressivity, and the exchange survives removal of the ceiling confound that made 6e-6h unreadable. Across capacity, the perplexity difference goes +1.82% -; +1.78% and the verifiability gain goes 83.2x -; 879.3x from `d_model` 32 to 64. Both cells are unsaturated, so unlike 6g this is a like-for-like comparison. Scope. This is a 2-layer model with an 8-character context on a 1 MB corpus. It is a

Extended Data Table 13. Expressivity cost of a data-independent normalizer on natural text. Character-level language modelling on tinyshakespeare (1 115 394 characters, 65-symbol vocabulary), context $T = 8$, 5 seeds per cell, 3000 steps. Perplexity is raw and held out on 131 072 next-character predictions; unlike the accuracy tasks it has no ceiling, so no cell is confounded by saturation. The gap is the post-hoc hybrid-zonotope bound width divided by the attained width at $\rho = 0.0001$.

d_{model}	variant	held-out ppl	SEM	gap	vs. standard
32	standard	7.3039	0.0309	95.500	–
32	fixnorm	7.4369	0.0295	1.147	penalty
64	standard	6.6971	0.0165	1053.245	–
64	fixnorm	6.8164	0.0162	1.198	penalty

Perplexity penalty 1.82% to 1.78%; verifiability gain $83.2\times$ to $879.3\times$ across $d_{\text{model}} = 32$ to 64. Maximum containment violation $6.4\text{e-}14$ (float noise; the bound is sound on every run).

language-modelling task in the sense that matters here – no accuracy ceiling, so the measurement stays interpretable as capacity grows – and in no other sense. Nothing about frontier-scale language models follows. Method. Strictly post-hoc: no differentiable verification penalty was used, `src/bounds.py` is unmodified, and the gaps come from the same sound NumPy hybrid-zonotope engine as 6d-6h, at $\rho=0.0001$ on 4 perturbation directions at the final residual-stream position. Soundness held on every run: maximum containment violation $6.4\text{e-}14$ (float noise; a positive value here would mean an unsound bound). There is no safety-margin readout in this section – the unsafe-token designation was a property of the synthetic task, and no character of English has a counterpart, so the column is dropped rather than invented.

4.19 Depth and width: where the certifiable region ends (6j)

Extending the sweep to $L = 4$ (Table `tab:depthwidth`) separates the two variants categorically rather than by ratio. Across $d_{\text{model}} = 64, 128, 256$ the standard arm’s median gap runs $1.25\text{e}+10$, $8.11\text{e}+15$, $5.82\text{e}+22$ with 312, 1026 and 2075 unstable ReLUs; every one of those bounds is above the $1\text{e}+03$ informativeness threshold, so the standard model is not certifiable anywhere on the row. The fixnorm arm stays below it at every width with a median of zero unstable ReLUs. No gain ratio is quoted, because a ratio whose denominator is itself uninformative is not a measurement. The fixnorm row is 1.67, 213.2, 291.7. That is a factor 128 between $d = 64$ and $d = 128$ and a factor 1.37 between $d = 128$ and $d = 256$. The comparable $L = 2$ row is 1.18, 2.38, 2.41, a factor 2.0 across the same fourfold width change. Three widths are enough to say the $L = 4$ curve is inconsistent with a smooth power law and to place a large step between the first two widths; they are not enough to localise where that step begins, and no width exponent is fitted to them. The earlier c18 exponents (standard 11.12, fixnorm 2.46) were fitted at $L = 2$ on random initialisation and should not be extrapolated to depth. Two cells on this row have a perplexity comparison that does not hold: ($L = 4$,

Extended Data Table 14. Depth and width scaling of the relaxation gap on character-level TinyShakespeare, $T = 8$, 3 seeds per cell, 3000 steps, $\rho = 1\text{e-}4$. Gap is the post-hoc hybrid-zonotope bound width over the attained width; medians, because run-to-run spread at fixed configuration reaches $2.4\times$. Cells marked – were not verified: $d = 512$ lies outside the verification subgrid and $L = 12$, $d = 512$ diverged on every seed. A gap above $1\text{e}3$ means the bound is not informative, so no gain ratio is quoted. The pairing column flags cells whose two arms trained on different devices, where the measured CUDA/CPU perplexity drift (0.62%) is comparable to the penalty being reported and the perplexity comparison is therefore not valid.

L	d_{model}	standard gap	unstable	fixnorm gap	unstable	ppl pairing
2	64	1.16e3	2	1.18	0	valid
2	128	2.07e4	37	2.38	0	valid
2	256	5.55e4	40	2.41	0	valid
2	512	–	–	–	–	valid
4	64	1.25e10	312	1.67	0	cross-device
4	128	8.11e15	1026	213	0	valid
4	256	5.82e22	2075	292	0	cross-device
12	512	–	–	–	–	incomplete

The fixnorm row at $L = 4$ reads 1.67, 213.2, 291.7 across $d = 64, 128, 256$: a factor 128 between the first two widths and 1.37 between the last two. Three widths are consistent with an onset between $d = 64$ and $d = 128$ and are inconsistent with a smooth power law, but they cannot localise the onset; no width exponent is fitted.

$d = 64$) and ($L = 4$, $d = 256$) trained their two arms on different devices. Measured CUDA-versus-CPU drift at $L = 4$, $d = 128$ fixnorm is 6.4245 against 6.3851, about 0.62% of baseline, which is the same order as the 1 to 2.5% perplexity penalties being reported. The gap column is unaffected: the two arms there differ by twenty orders of magnitude and a 0.6% drift cannot move that conclusion. Only the $L = 4$, $d = 128$ penalty is quoted as a valid pairing.

4.20 Optimising the bound during training collapses its variance (6k)

Sections 6d-6j measure a bound after training. This section puts an interval-bound (IBP) upper bound on the loss into the objective itself, with the standard kappa/epsilon ramp, and asks what changes. The training-time bound is plain interval arithmetic and is far looser than the hybrid zonotope; it is a training signal only. Every gap reported here is still measured post hoc by the same sound NumPy prover used throughout, so the two arms are certified by one engine. Note the radius differs from 6d-6j: certification here is at $\rho = 0.02$, where ordinary training frequently fails to certify at all, rather than the $\rho = 10^{-4}$ used earlier. At $L = 2$, $d_{\text{model}} = 32$, pooled over three learning rates, the control’s fixnorm gap spans 2.51 to 4.27e+03, a factor of 1705 across $n = 9$ runs, and only 6 of those 9 fall below the informativeness threshold. The certified arm spans 1.13 to 1.22, a factor of 1.08 across $n = 8$ runs, all 8 informative, with zero unstable ReLUs on every run. The primary effect is therefore

on the SPREAD, not on the median alone: ordinary training at this radius produces a bound whose quality is a lottery across seeds, and certified training produces the same bound every time. The median ratio is deliberately not quoted. Its denominator moves by a factor of 1705 across seeds, so the ratio is a statement about which seed the control happened to draw. A single-seed version of this experiment reported 2.08x at one learning rate where three seeds give a median control gap three orders of magnitude higher; that discrepancy is the reason the ratio is withheld and the spread is reported instead. The exchange is paid in perplexity: 13.3% to 28.0% above the matched control across the three learning rates. That is an order of magnitude larger than the 1 to 2.5% cost of the fixed normaliser alone. Two facts bound how far this generalises, and they cut in opposite directions. The standard variant diverged on all three seeds at all 3 learning rates, so it has no certified arm to compare against even at $L = 2$, $d_{\text{model}} = 32$. And the failure is not confined to that one variant: all 18 of the 18 other cell configurations attempted under active certified training – spanning both the standard and fixnorm variants at $d = 64$ and $L = 4$ – diverged at every learning rate tested, under the one fixed epsilon-ramp schedule used throughout this grid. The supportable claim is therefore narrow: at the one cell where certified training converges at all, it converges for fixnorm and not for standard, and it makes the resulting bound reproducible. These data do not show that a fixed normaliser is a prerequisite for certified training, and they do not show that certified training scales past the smallest configuration tested. Locating the epsilon at which $d = 64$ becomes trainable was tested directly rather than left open. Three deliberately gentler schedules were run at $L = 2$, $d_{\text{model}} = 64$, fixnorm, three seeds each: a smaller target radius ($\epsilon = 0.02$, same 100+1000 ramp) (3/3 seeds diverged); the original target radius on a much slower ramp (200+2500) (3/3 seeds diverged); both combined ($\epsilon = 0.02$, 200+2500 ramp) (3/3 seeds diverged). All 9 runs diverged. This rules out the two most obvious fixes, a smaller target and a slower approach to it, without ruling out every possible schedule in a space too large to search exhaustively here; the supportable claim stays that certified training reproduces one configuration, not that a schedule search would fail in general.

4.21 An honest adversarial stress test, and what it does and does not show (6l)

Section 6k’s bound is not differentiable – it comes from the NumPy hybrid-zonotope prover, a discrete branch-and-bound search, not a computation graph – so a gradient-based adversary cannot attack it directly. What IS differentiable is `src/torch.bounds.py`, the plain interval-arithmetic engine used as the certified-training signal in 6k. This section trains a Gumbel-Softmax relaxation over the $L = 2$, $d_{\text{model}} = 32$ certified fixnorm cell’s `SEQ\{\}_LEN` context tokens to maximise that engine’s bound width by gradient ascent, snaps to the discrete argmax context, and re-evaluates exactly (no relaxation) on the snapped tokens. This is a search for slack in the training signal, not an attack on the certificate itself, and a failure to find a large gap would demonstrate only that this search failed, not that no gap exists. Over 5 independent restarts of 300 steps each, the search converged on repeated-token, out-of-distribution contexts (e.g. `zQQQQQ.`) with a torch-engine relaxation gap of 107

Extended Data Table 15. Certified training versus ordinary training, $L = 2$, $d_{\text{model}} = 32$, $\rho = 0.02$, 3000 steps, 3 seeds per arm. Control is the identical recipe with the robust term disabled. The certified arm optimises an interval-bound upper bound on the loss; the gap column is still measured post hoc by the hybrid-zonotope prover, so the two arms are certified by the same engine. Standard diverges under certified training at every learning rate, so it has no certified arm.

lr	variant	arm	converged	ppl (SEM)	gap median	gap range
0.001	fixnorm	control	3/3	8.463 (0.007)	19.5	14.8–76.3
0.001	fixnorm	certified	3/3	10.830 (0.060)	1.19	1.13–1.22
0.001	standard	control	3/3	8.181 (0.008)	1.94e8	5.84e5–4.4e8
0.001	standard	certified	0/3	–	–	diverged
0.003	fixnorm	control	3/3	7.994 (0.036)	1.27e3	2.51–1.65e3
0.003	fixnorm	certified	3/3	9.889 (0.029)	1.19	1.14–1.2
0.003	standard	control	3/3	7.890 (0.018)	1.39e9	1.29e9–3.19e9
0.003	standard	certified	0/3	–	–	diverged
0.01	fixnorm	control	3/3	8.051 (0.067)	49.3	8.26–4.27e3
0.01	fixnorm	certified	2/3	9.118 (0.044)	1.13	1.13–1.13
0.01	standard	control	3/3	7.931 (0.064)	1.85e8	1.09e7–1.42e10
0.01	standard	certified	0/3	–	–	diverged

Pooled over all three learning rates, the control’s fixnorm gap spans 2.51 to 4.27e3 (1705 $\times$, $n = 9$, of which 6 fall below the 1e3 informativeness threshold), while the certified arm spans 1.13 to 1.22 (1.08 $\times$, $n = 8$, all 8 informative). The effect is a collapse of spread, not only of the median; the median ratio itself is withheld because its denominator is unstable.

(median across restarts, max 113), against a median of 6.69 on 8 natural TinyShakespeare contexts – a 16.9x increase, reproduced in 5 of 5 restarts rather than appearing once. This is a real, repeatable OOD sensitivity in the interval-arithmetic training signal, consistent with IBP’s known looseness away from the calibration distribution. Whether this transfers to the certificate that is actually reported is a separate question, checked here black-box (the zonotope prover was never part of the gradient search): the same adversarial context, evaluated by the unmodified NumPy prover at the real certified $\rho = 0.02$, gives a gap of 1.15 against 1.09 on a natural context – a 1.05x shift, not the 17x found in the training-signal engine. Both remain far inside the certifiable band, both show zero unstable ReLUs, and containment holds at both points (worst violation 8.9e-15, float noise, consistent with the containment checks elsewhere in this project). The interpretation offered is that the correlation-preserving zonotope – which keeps first-order cross-terms across the softmax product exactly, unlike the interval engine (section 1) – is why the OOD slack found in the training signal does not carry over to the certifying engine on this one context. This is one context pair, not a swept claim, and it is not evidence that no context defeats the zonotope prover, only that this particular search, run against this particular engine, did not find one that does. Scope, stated plainly: only the fixnorm+ReLU architecture was attacked, because it is the only architecture with an actual banked NumPy-certified cell – the golden-section GeLU/SiLU engine in `src/torch_bounds.py` has never been NumPy-zonotope certified (`arch.certifiable()` skips it by construction) and attacking it would be attacking a code path nothing in this manuscript has claimed a certificate for.

The search ran on one trained seed, one learning rate, one radius, and one temperature schedule. None of this is a proof of immunity, in either direction.

4.22 Limitations

- This is a 26,144-parameter toy transformer on a synthetic task. Nothing has been shown to scale, and the unstable-ReLU mechanism worsens with width and depth.
- The empirical radius is an UPPER bound on the true robust radius. PGD breaking at $\rho=10$ shows the true radius is at most 10; PGD failing at $\rho=3$ shows nothing, because PGD is incomplete. The true radius lies in (certified, 10]. The conservativeness gap is therefore 'at most 250x', and it is a property of this prover on this model, not a general tax for formal guarantees.
- The end-to-end bound has no independent confirmation (section 4).
- Safety is an unsafe-logit margin over a designated token subset.
- The certificate covers a fixed discrete context; only the continuous perturbation is quantified over.
- The language-modelling evidence in 6i removes the accuracy ceiling that confounded 6e-6h, but it does not add scale: 2 layers, an 8-character context, and a 1 MB corpus. It licenses claims about the penalty-versus-verifiability exchange being measurable on natural text, not claims about language models.
- 6i reports no safety margin. The unsafe-logit readout that carries sections 1-6h has no counterpart in text, so the verifiability half of 6i is a relaxation gap only, not a certified safety property.
- 6l's adversarial search only reaches the differentiable torch IBP engine, never the NumPy zonotope prover directly; the zonotope result there is a single black-box context pair, not a swept adversarial evaluation of the certifying engine itself.

4.23 Methods and Reproduction

Python 3.14.3, numpy 2.4.2, torch 2.10.0+cpu, scipy 1.16.3, z3 4.15.6, cvxpy 1.9.2. dReal absent. Seed 0 throughout; `results/environment.json` records every seed and a SHA-256 for each checkpoint and result file.

<code>python stages/a0_bootstrap.py</code>	<code># full pipeline -> results/baseline.json</code>
<code>python audit/c14_crosscheck.py</code>	<code># falsification + coverage</code>
<code>python audit/z3_primitives.py</code>	<code># exact SMT on the primitives</code>
<code>python audit/sweep_layernorm.py</code>	<code># bracket cliff</code>
<code>python audit/c16_experiments.py</code>	<code># prompt distribution + branching</code>
<code>python audit/crown_stagewise.py</code>	<code># CROWN vs prover, per stage</code>
<code>python audit/c23_language.py</code>	<code># char-LM penalty vs gap -> results/c23.json</code>
<code>python audit/c29_fuzzer.py</code>	<code># adversarial stress test -> results/c29_fuzz.json</code>
<code>python audit/verify_bundle.py</code>	<code># gate: presence, sync, hashes</code>